EXPERIMENT: BREADTH FIRST SEARCH (BFS)

Aim

To write a Python program to implement the Breadth First Search (BFS) algorithm.

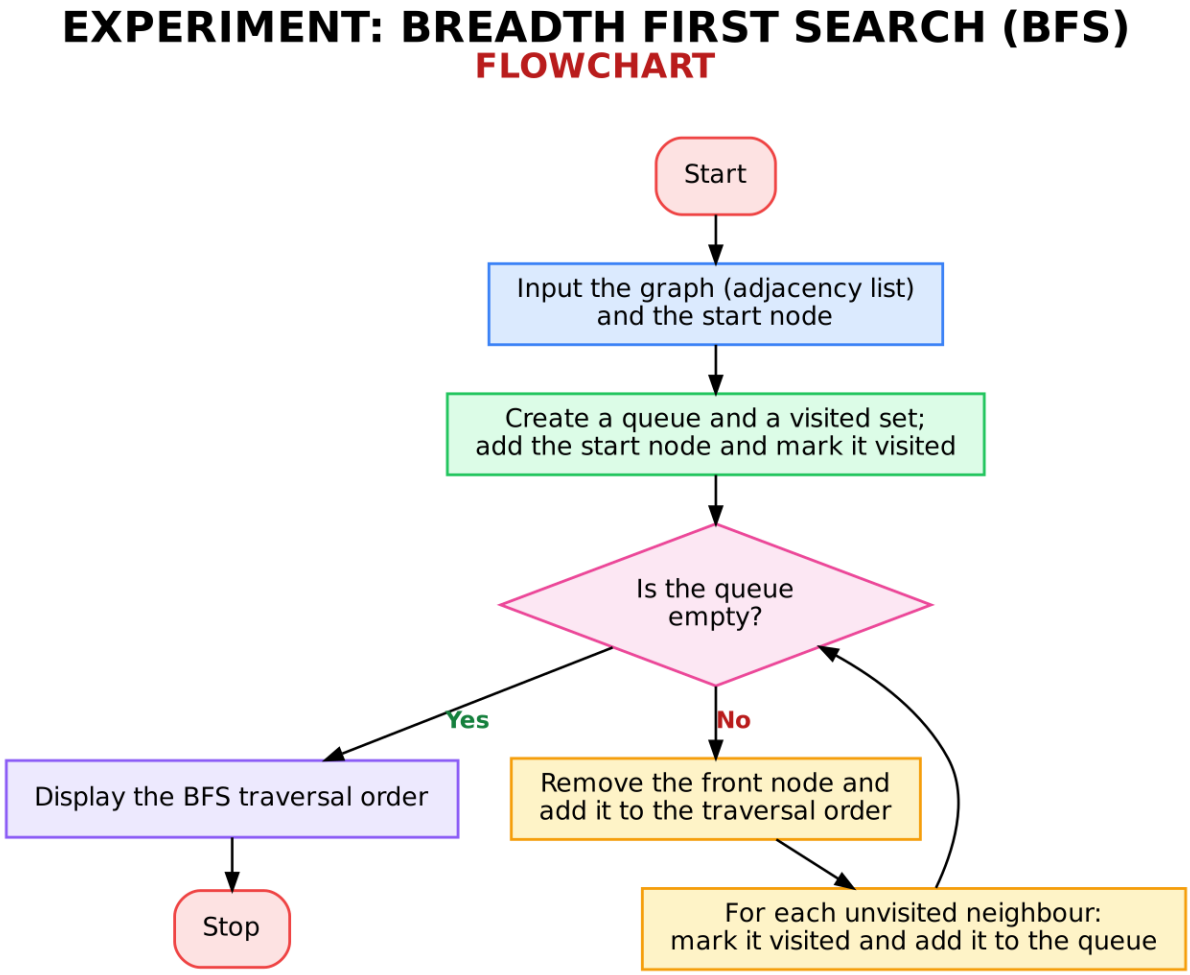
Objective

- To understand the representation of a graph using an adjacency list.
- To implement a queue-based level-by-level traversal of a graph.
- To learn how BFS visits all the nodes reachable from a start node.

Algorithm

1. Start.
2. Represent the graph as an adjacency list and choose the start node.
3. Create an empty queue and an empty visited set.
4. Insert the start node into the queue and mark it as visited.
5. While the queue is not empty, remove the front node and add it to the traversal order.
6. Visit all unvisited neighbours of that node, mark them as visited and insert them into the queue.
7. Repeat steps 5 and 6 until the queue becomes empty.
8. Display the traversal order.
9. Stop.

Flowchart:



Code (Python)

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return order

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
```

```
    'D': [],
    'E': ['F'],
    'F': []
}

print("BFS traversal starting from A:")
print(bfs(graph, 'A'))
```

Output

Python Shell - Output

```
BFS traversal starting from A:
['A', 'B', 'C', 'D', 'E', 'F']
```

Output Result

Thus, the Python program to implement Breadth First Search was executed successfully, and the nodes were visited level by level as A, B, C, D, E, F.

Conclusion

The experiment helped in understanding graph traversal. Breadth First Search uses a queue to visit the nodes level by level and is useful for finding the shortest path in an unweighted graph.